LAPORAN TUGAS PRAKTIKUM
Implementasi Function dan Trigger pada Sistem Penjualan Toko Elektronik

Disusun Untuk Memenuhi Tugas Mata Kuliah

Rekayasa & Arsitektur Basis Data

Dosen Pengampu:

Cecep Kurnia Sastradipraja, S.Kom., M.Kom.

Disusun Oleh:

M Akbar Raditya P 2507070

PROGRAM STUDI SISTEM INFORMASI INSTITUT TEKNOLOGI GARUT

2026

1.1 Tujuan Praktikum

Praktikum ini bertujuan agar mahasiswa mampu membuat basis data dan tabel relasional, membuat stored function untuk menghitung total transaksi dan diskon, membuat trigger untuk validasi transaksi dan pembaruan stok, serta menguji seluruhnya melalui skenario transaksi valid dan tidak valid menggunakan MySQL 8.0 (DBngin) dan DBeaver.

1.2 Deskripsi Kasus

Sebuah toko elektronik ingin mengotomatisasi penjualan dan pengendalian persediaan. Setiap produk memiliki ID, nama, harga, dan stok. Saat transaksi terjadi, sistem harus menghitung total harga, memvalidasi stok, mengurangi stok, dan mencatat perubahan stok dalam log.

1.3 Rancangan Solusi

Objek	Jenis	Fungsi
produk, penjualan, log_stok	Tabel	Menyimpan data produk, transaksi, dan riwayat stok
hitung_total	Function	Total = harga satuan x jumlah
hitung_diskon	Function	Diskon 10% jika total minimal Rp1.000.000
validasi_penjualan	Trigger BEFORE INSERT pada penjualan	Validasi jumlah, harga, produk, stok; mengisi kolom total
update_stok_penjualan	Trigger AFTER INSERT pada penjualan	Mengurangi stok dan menulis log_stok
hitung_pajak, hitung_total_akhir	Function (tugas mandiri)	Pajak 11% dan total akhir setelah diskon persentase
log_perubahan_stok	Trigger AFTER UPDATE pada produk (mandiri)	Mencatat setiap perubahan stok
validasi_harga_produk	Trigger BEFORE UPDATE pada produk (mandiri)	Menolak harga nol atau negatif

Alur transaksi: INSERT ke penjualan, lalu trigger BEFORE memvalidasi dan menghitung total memakai hitung_total, baris disimpan, lalu trigger AFTER mengurangi stok dan mencatat log.

2. Persiapan Lingkungan

1. Buka DBngin, buat instance MySQL 8.0, lalu jalankan hingga statusnya aktif. Catat host, port, username, dan password.
2. Buka DBeaver, pilih Database, New Database Connection, pilih driver MySQL, isi host, port, username, password, lalu Test Connection dan Finish.

3. Buka SQL Editor. Skrip yang memuat DELIMITER dijalankan dengan Execute SQL Script (Alt+X), bukan eksekusi satu pernyataan.

```
SELECT VERSION() AS versi_mysql, DATABASE() AS basis_data_aktif;
```

Kolom versi_mysql menampilkan versi server (8.0.x). Kolom basis_data_aktif bernilai NULL karena basis data belum dipilih.

[Tempel screenshot hasil koneksi dan query versi di sini]

3. Praktikum 1: Membuat Basis Data dan Tabel

3.1 Membuat dan memilih basis data

```
CREATE DATABASE praktikum_function_trigger;
USE praktikum_function_trigger;
SELECT DATABASE();
```

CREATE DATABASE membuat basis data baru, USE menjadikannya basis data aktif, dan SELECT DATABASE() memastikan basis data yang aktif adalah praktikum_function_trigger.

3.2 Tabel produk

```
CREATE TABLE produk (
   id_produk   INT AUTO_INCREMENT PRIMARY KEY,
   nama_produk VARCHAR(100) NOT NULL,
   harga       DECIMAL(12,2) NOT NULL,
   stok        INT NOT NULL DEFAULT 0,
   CHECK (harga > 0),
   CHECK (stok >= 0)
) ENGINE = InnoDB;
```

id_produk adalah kunci utama yang naik otomatis. Constraint CHECK menjaga harga selalu positif dan stok tidak pernah negatif. Engine InnoDB dipilih karena mendukung transaksi, foreign key, dan penguncian baris.

3.3 Tabel penjualan

```
CREATE TABLE penjualan (
   id_penjualan      INT AUTO_INCREMENT PRIMARY KEY,
   id_produk         INT NOT NULL,
   jumlah            INT NOT NULL,
   harga_satuan      DECIMAL(12,2) NOT NULL,
   total             DECIMAL(14,2) NOT NULL DEFAULT 0,
   tanggal_transaksi TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   CONSTRAINT fk_penjualan_produk
      FOREIGN KEY (id_produk) REFERENCES produk(id_produk),
   CHECK (jumlah > 0),
   CHECK (harga_satuan > 0),
   CHECK (total >= 0)
) ENGINE = InnoDB;
```

Foreign key fk_penjualan_produk memastikan transaksi hanya dapat mengacu pada produk yang ada. Kolom tanggal_transaksi terisi otomatis dengan waktu saat data dimasukkan.

3.4 Tabel log_stok

```
CREATE TABLE log_stok (
   id_log     INT AUTO_INCREMENT PRIMARY KEY,
   id_produk  INT NOT NULL,
   stok_awal  INT NOT NULL,
   stok_akhir INT NOT NULL,
   keterangan VARCHAR(100) NOT NULL,
   waktu_log  TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE = InnoDB;
```

Tabel ini menyimpan riwayat perubahan stok (nilai sebelum, nilai sesudah, alasan, dan waktu).

3.5 Memasukkan data awal

```
INSERT INTO produk (nama_produk, harga, stok)
VALUES
   ('Laptop', 7500000, 10),
   ('Mouse', 150000, 25),
   ('Keyboard', 350000, 15),
   ('Monitor', 1800000, 8);

SELECT * FROM produk;
```

Hasil:

id_produk	nama_produk	harga	stok
1	Laptop	7500000.00	10
2	Mouse	150000.00	25
3	Keyboard	350000.00	15
4	Monitor	1800000.00	8

[Tempel screenshot DESC dan SELECT tabel produk di sini]

4. Praktikum 2: Stored Function

4.1 Function hitung_total

```
DELIMITER //
CREATE FUNCTION hitung_total(
   p_harga  DECIMAL(12,2),
   p_jumlah INT
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
   RETURN p_harga * p_jumlah;
END //
```

```
DELIMITER ;

SELECT hitung_total(150000, 3) AS total_harga;
```

Penjelasan: DELIMITER // mengganti pemisah pernyataan sementara agar titik koma di dalam BEGIN...END tidak mengakhiri CREATE FUNCTION. DETERMINISTIC berarti input yang sama selalu menghasilkan output yang sama, dan NO SQL berarti function tidak membaca atau mengubah tabel. Hasil: total_harga = 450000.00.

4.2 Function hitung_diskon

```
DELIMITER //
CREATE FUNCTION hitung_diskon(
   p_total DECIMAL(14,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
   IF p_total >= 1000000 THEN
      RETURN p_total * 0.10;
   ELSE
      RETURN 0;
   END IF;
END //
DELIMITER ;

SELECT
   1500000 AS total_transaksi,
   hitung_diskon(1500000) AS diskon,
   1500000 - hitung_diskon(1500000) AS total_bayar;
```

Penjelasan: percabangan IF memberi diskon 10% bila total minimal Rp1.000.000, selain itu diskon 0.

total_transaksi	diskon	total_bayar
1500000	150000.00	1350000.00

4.3 Menggabungkan function

```
SELECT
   hitung_total(350000, 4) AS total_transaksi,
   hitung_diskon(hitung_total(350000, 4)) AS diskon,
   hitung_total(350000, 4)
     - hitung_diskon(hitung_total(350000, 4)) AS total_bayar;
```

Penjelasan: hasil hitung_total langsung dijadikan input hitung_diskon.

total_transaksi	diskon	total_bayar
1400000.00	140000.00	1260000.00

[Tempel screenshot pengujian function di sini]

5. Praktikum 3: Database Trigger

5.1 Trigger validasi_penjualan (BEFORE INSERT)

```
DELIMITER //
CREATE TRIGGER validasi_penjualan
BEFORE INSERT ON penjualan
FOR EACH ROW
BEGIN
   DECLARE v_stok INT DEFAULT NULL;
   DECLARE v_ada  INT DEFAULT 0;

   IF NEW.jumlah <= 0 THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'Jumlah pembelian harus lebih dari nol';
   END IF;

   IF NEW.harga_satuan <= 0 THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'Harga satuan harus lebih dari nol';
   END IF;

   SELECT COUNT(*) INTO v_ada
   FROM produk WHERE id_produk = NEW.id_produk;

   IF v_ada = 0 THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'Produk tidak ditemukan';
   END IF;

   SELECT stok INTO v_stok
   FROM produk WHERE id_produk = NEW.id_produk
   FOR UPDATE;

   IF NEW.jumlah > v_stok THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'Stok produk tidak mencukupi';
   END IF;

   SET NEW.total = hitung_total(NEW.harga_satuan, NEW.jumlah);
END //
DELIMITER ;
```

Penjelasan tahap demi tahap:

1. Trigger berjalan sebelum baris disimpan. NEW menyatakan baris yang akan dimasukkan.
2. Dua pemeriksaan pertama menolak jumlah atau harga satuan yang nol atau negatif.

3. COUNT(*) memeriksa keberadaan produk agar pesan kesalahan lebih jelas daripada kegagalan foreign key.
4. SELECT ... FOR UPDATE membaca stok sekaligus mengunci baris produk selama transaksi berlangsung sehingga tidak ada transaksi lain yang mengubah stok di tengah pemeriksaan.
5. Bila jumlah melebihi stok, SIGNAL SQLSTATE '45000' menghentikan INSERT dan seluruh perubahannya dibatalkan.
6. Jika semua valid, SET NEW.total mengisi kolom total memakai function hitung_total. Nilai total yang dikirim pengguna (0) diabaikan.

5.2 Trigger update_stok_penjualan (AFTER INSERT)

```
DELIMITER //
CREATE TRIGGER update_stok_penjualan
AFTER INSERT ON penjualan
FOR EACH ROW
BEGIN
   DECLARE v_stok_awal  INT;
   DECLARE v_stok_akhir INT;

   SELECT stok INTO v_stok_awal
   FROM produk WHERE id_produk = NEW.id_produk;

   SET v_stok_akhir = v_stok_awal - NEW.jumlah;

   UPDATE produk SET stok = v_stok_akhir
   WHERE id_produk = NEW.id_produk;

   INSERT INTO log_stok (id_produk, stok_awal, stok_akhir, keterangan)
   VALUES (NEW.id_produk, v_stok_awal, v_stok_akhir,
         'Pengurangan stok karena penjualan');
END //
DELIMITER ;
```

Penjelasan: trigger berjalan setelah baris penjualan tersimpan. Stok saat ini dibaca sebagai stok awal, stok akhir dihitung (stok awal dikurangi jumlah), tabel produk diperbarui, lalu satu baris dicatat ke log_stok.

5.3 Pengujian transaksi valid (2 unit Mouse)

```
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (2, 2, 150000, 0);

SELECT * FROM penjualan;
SELECT nama_produk, stok FROM produk WHERE id_produk = 2;
SELECT * FROM log_stok;
```

Hasil: kolom total terisi 300000.00 (bukan 0) oleh trigger BEFORE, stok Mouse menjadi 23, dan log mencatat stok awal 25 dan stok akhir 23 dengan keterangan Pengurangan stok karena penjualan.

5.4 Pengujian transaksi tidak valid (100 unit Mouse)

```
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (2, 100, 150000, 0);
```

Hasil: ditolak dengan pesan SQL Error [1644] [45000]: Stok produk tidak mencukupi. Tabel penjualan, stok Mouse (tetap 23), dan log_stok tidak berubah karena trigger BEFORE menghentikan pernyataan sebelum baris disimpan.

5.5 Memeriksa function dan trigger

```
SHOW FUNCTION STATUS WHERE Db = 'praktikum_function_trigger';
SHOW TRIGGERS FROM praktikum_function_trigger;
```

Hasil: SHOW FUNCTION STATUS menampilkan hitung_total dan hitung_diskon, dan SHOW TRIGGERS menampilkan validasi_penjualan (BEFORE INSERT) dan update_stok_penjualan (AFTER INSERT) pada tabel penjualan.

[Tempel screenshot pengujian trigger dan SHOW TRIGGERS di sini]

6. Tugas Terstruktur (Dikerjakan di Kelas)

Tugas 1. Function Perhitungan Total

```
SELECT hitung_total(250000, 4) AS total_transaksi;
```

Hasil: total_transaksi = 1000000.00 (Rp1.000.000), sesuai target (250.000 x 4).

Tugas 2. Function Diskon

```
SELECT 'A' AS pengujian,
    800000 AS total_transaksi,
    hitung_diskon(800000) AS diskon,
    800000 - hitung_diskon(800000) AS total_bayar
UNION ALL
SELECT 'B',
    1500000,
    hitung_diskon(1500000),
    1500000 - hitung_diskon(1500000);
```

pengujian	total_transaksi	diskon	total_bayar
A	800000	0.00	800000.00
B	1500000	150000.00	1350000.00

Analisis: pengujian A (Rp800.000) di bawah batas Rp1.000.000 sehingga tidak ada diskon. Pengujian B memenuhi syarat sehingga mendapat diskon 10% (Rp150.000).

Tugas 3. Validasi Transaksi Penjualan

Setiap skenario dijalankan terpisah (satu pernyataan per eksekusi, Ctrl+Enter).

Skenario A, Mouse 1 unit (diterima):

```
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (2, 1, 150000, 0);
```

Skenario B, Mouse 100 unit (ditolak):

```
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (2, 100, 150000, 0);
```

Skenario C, Mouse 0 unit (ditolak):

```
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (2, 0, 150000, 0);
```

Skenario	Produk	Jumlah	Hasil	Pesan
A	Mouse	1	Diterima	1 row affected; total terisi 150000.00; stok Mouse 23 menjadi 22
B	Mouse	100	Ditolak	Stok produk tidak mencukupi
C	Mouse	0	Ditolak	Jumlah pembelian harus lebih dari nol

Analisis: trigger validasi_penjualan hanya meloloskan transaksi yang jumlahnya positif dan tidak melebihi stok. Pada skenario C, trigger BEFORE berjalan lebih dulu daripada CHECK constraint sehingga pesan yang muncul adalah pesan kustom dari trigger.

Tugas 4. Pembaruan dan Pencatatan Stok (3 unit Keyboard)

```
-- Stok awal
SELECT id_produk, nama_produk, stok FROM produk WHERE id_produk = 3;

-- Transaksi
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (3, 3, 350000, 0);

-- Stok akhir
SELECT id_produk, nama_produk, stok FROM produk WHERE id_produk = 3;

-- Log
SELECT * FROM log_stok WHERE id_produk = 3;
```

Pemeriksaan	Hasil
Stok awal Keyboard	15
Total transaksi	1050000.00
Stok akhir Keyboard	12
log_stok	stok_awal 15, stok_akhir 12, keterangan Pengurangan stok karena penjualan

Analisis: trigger update_stok_penjualan mengurangi stok 15 menjadi 12 dan mencatatnya ke log_stok tanpa perintah UPDATE manual.

Tugas 5. Integrasi Function dan Trigger (2 unit Laptop)

```
-- Stok sebelum transaksi
SELECT nama_produk, stok AS stok_sebelum FROM produk WHERE id_produk = 1;

-- Transaksi
INSERT INTO penjualan (id_produk, jumlah, harga_satuan, total)
VALUES (1, 2, 7500000, 0);

-- Total, diskon, dan total bayar
SELECT p.id_penjualan,
      pr.nama_produk,
      p.jumlah,
      p.total AS total_transaksi,
      hitung_diskon(p.total) AS diskon,
      p.total - hitung_diskon(p.total) AS total_bayar
FROM penjualan p
JOIN produk pr ON pr.id_produk = p.id_produk
WHERE p.id_produk = 1;

-- Stok sesudah transaksi
SELECT nama_produk, stok AS stok_sesudah FROM produk WHERE id_produk = 1;

-- Log stok
SELECT * FROM log_stok WHERE id_produk = 1;
```

Komponen	Hasil
Total transaksi	Rp15.000.000 (2 x 7.500.000)
Diskon 10%	Rp1.500.000
Total pembayaran	Rp13.500.000
Stok sebelum	10
Stok sesudah	8
Log stok	stok_awal 10, stok_akhir 8, Pengurangan stok karena penjualan

Analisis: satu INSERT memicu seluruh rantai otomatis. Trigger BEFORE memvalidasi dan menghitung total lewat hitung_total, function hitung_diskon menghitung diskon dari total tersebut, lalu trigger AFTER memperbarui stok dan mencatat log.

[Tempel screenshot hasil eksekusi Tugas 1 sampai 5 di sini]

Kondisi data setelah tugas terstruktur: Laptop 8, Mouse 22, Keyboard 12, Monitor 8. Tabel log_stok berisi empat catatan: Mouse 25 ke 23, Mouse 23 ke 22, Keyboard 15 ke 12, dan Laptop 10 ke 8.

7. Tugas Mandiri

Tugas 1. Function hitung_pajak

```
DELIMITER //
CREATE FUNCTION hitung_pajak(
   p_total DECIMAL(14,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
   RETURN p_total * 0.11;
END //
DELIMITER ;

SELECT 2000000 AS total_transaksi, hitung_pajak(2000000) AS pajak
UNION ALL
SELECT 5000000, hitung_pajak(5000000);
```

total_transaksi	pajak	Diharapkan
2000000	220000.00	Rp220.000 (sesuai)
5000000	550000.00	Rp550.000 (sesuai)

Penjelasan: function memakai pola yang sama dengan hitung_diskon, yaitu satu parameter total dan hasil total dikali 11%.

Tugas 2. Function hitung_total_akhir

Rumus: Total Akhir = Total Transaksi - (Total Transaksi x Persentase Diskon / 100)

```
DELIMITER //
CREATE FUNCTION hitung_total_akhir(
   p_total         DECIMAL(14,2),
   p_persen_diskon  DECIMAL(5,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
   RETURN p_total - (p_total * p_persen_diskon / 100);
END //
DELIMITER ;

SELECT hitung_total_akhir(3000000, 15) AS total_akhir;
```

Hasil: 3.000.000 - (3.000.000 x 15 / 100) = 2550000.00 (Rp2.550.000), sesuai target. Function memiliki dua parameter: total dan persentase diskon.

Tugas 3. Trigger AFTER UPDATE pencatatan stok

Persiapan: stok Mouse saat ini 22 akibat praktikum sebelumnya, maka diatur kembali ke 25. Perintah ini dijalankan sebelum trigger dibuat agar tidak ikut tercatat di log.

```
UPDATE produk SET stok = 25 WHERE id_produk = 2;

DELIMITER //
CREATE TRIGGER log_perubahan_stok
AFTER UPDATE ON produk
FOR EACH ROW
BEGIN
   IF OLD.stok <> NEW.stok THEN
      INSERT INTO log_stok (id_produk, stok_awal, stok_akhir, keterangan)
      VALUES (NEW.id_produk, OLD.stok, NEW.stok,
            'Perubahan stok pada tabel produk');
   END IF;
END //
DELIMITER ;

-- Pengujian: stok Mouse 25 menjadi 30
UPDATE produk SET stok = 30 WHERE id_produk = 2;

SELECT * FROM log_stok WHERE id_produk = 2 ORDER BY id_log DESC LIMIT 1;
```

id_produk	stok_awal	stok_akhir	keterangan	waktu_log
2	25	30	Perubahan stok pada tabel produk	(waktu eksekusi)

Penjelasan: OLD.stok adalah nilai sebelum UPDATE dan NEW.stok nilai sesudahnya. Kondisi IF OLD.stok <> NEW.stok memastikan log hanya dibuat saat stok benar-benar berubah, sehingga perubahan kolom lain seperti harga tidak mencatat log. Kolom waktu_log terisi otomatis oleh CURRENT_TIMESTAMP.

Catatan penting: trigger update_stok_penjualan juga menjalankan UPDATE pada produk, sehingga setelah log_perubahan_stok dibuat, setiap penjualan akan menghasilkan dua catatan log (satu dari trigger penjualan dan satu dari trigger ini). Karena itu pengujian mandiri dilakukan setelah seluruh tugas terstruktur selesai dan tanpa transaksi penjualan baru. Pada sistem nyata, salah satu pencatatan sebaiknya dihilangkan untuk menghindari duplikasi.

Tugas 4. Trigger BEFORE UPDATE validasi harga

```
DELIMITER //
CREATE TRIGGER validasi_harga_produk
BEFORE UPDATE ON produk
FOR EACH ROW
BEGIN
```

```
   IF NEW.harga <= 0 THEN
      SIGNAL SQLSTATE '45000'
         SET MESSAGE_TEXT = 'Harga produk harus lebih dari nol';
   END IF;
END //
DELIMITER ;

-- Skenario 1: diterima
UPDATE produk SET harga = 200000 WHERE id_produk = 2;

-- Skenario 2: ditolak
UPDATE produk SET harga = 0 WHERE id_produk = 2;

SELECT id_produk, nama_produk, harga FROM produk WHERE id_produk = 2;
```

Skenario	Perintah	Hasil
1	Harga Mouse menjadi 200000	Diterima, 1 row affected
2	Harga Mouse menjadi 0	Ditolak: SQL Error [1644] [45000]: Harga produk harus lebih dari nol

SELECT terakhir menunjukkan harga Mouse tetap 200000.00 karena perubahan ke 0 dibatalkan. Kondisi NEW.harga <= 0 juga mencakup harga negatif. Trigger ini berjalan sebelum constraint CHECK (harga > 0), sehingga pesan yang tampil adalah pesan kustom.

[Tempel screenshot hasil Tugas Mandiri 1 sampai 4 di sini]

Tugas 5. Analisis Integrasi Function dan Trigger

1. Perbedaan hitung_total dan validasi_penjualan. Function hitung_total dipanggil secara eksplisit, menerima parameter, dan mengembalikan satu nilai (hasil perkalian) tanpa mengubah data. Trigger validasi_penjualan berjalan otomatis ketika terjadi INSERT pada penjualan, tidak menerima parameter, tidak mengembalikan nilai, dan dapat memeriksa data, menolak operasi lewat SIGNAL, serta mengubah baris yang akan disimpan. Dalam praktikum ini trigger bahkan memanggil function untuk mengisi kolom total.
2. Dampak jika trigger pembaruan stok tidak dibuat. Transaksi tetap tersimpan di penjualan tetapi stok di produk tidak berkurang. Data stok menjadi tidak sesuai dengan kenyataan, produk yang sebenarnya habis tetap terlihat tersedia, validasi stok menjadi tidak akurat sehingga terjadi penjualan melebihi persediaan (overselling), dan tidak ada catatan perubahan stok.
3. Alasan perubahan stok perlu dicatat dalam tabel log. Log menjadi jejak audit: dapat ditelusuri kapan stok berubah, dari berapa menjadi berapa, dan karena apa. Hal ini membantu mendeteksi kesalahan atau kecurangan, merekonsiliasi selisih stok dengan stok fisik, dan menjadi bukti pertanggungjawaban data. Tanpa log, hanya nilai stok terakhir yang diketahui.
4. Peran function dan trigger dalam menjaga integritas data. Function memusatkan aturan bisnis (rumus total, diskon, pajak) di satu tempat sehingga hasil perhitungan konsisten di

semua query. Trigger menegakkan aturan di sisi basis data, bukan di aplikasi, sehingga validasi (jumlah positif, stok cukup, harga valid) dan sinkronisasi stok selalu berjalan apa pun sumber INSERT atau UPDATE-nya. Bersama constraint (PRIMARY KEY, FOREIGN KEY, CHECK), keduanya menjaga integritas entitas, referensial, dan domain, serta konsistensi antar tabel.

5. Hal yang perlu diperhatikan saat dua transaksi membeli produk berstok terbatas secara bersamaan. Risikonya adalah race condition: keduanya membaca stok yang sama (misalnya 1), keduanya lolos validasi, lalu stok menjadi negatif atau salah hitung (lost update). Hal yang perlu diperhatikan:

- Gunakan engine InnoDB dan jalankan penjualan di dalam transaksi (START TRANSACTION ... COMMIT).
- Gunakan penguncian baris. Pada praktikum ini SELECT stok ... FOR UPDATE di trigger validasi_penjualan mengunci baris produk sehingga transaksi kedua menunggu sampai transaksi pertama COMMIT atau ROLLBACK, lalu membaca stok terbaru dan ditolak bila tidak cukup.
- Kunci baru dilepas saat COMMIT atau ROLLBACK, jadi transaksi sebaiknya singkat agar tidak menghambat transaksi lain atau memicu deadlock.
- Constraint CHECK (stok >= 0) menjadi pengaman terakhir, dan pengurangan stok yang atomik (UPDATE ... WHERE stok >= jumlah) dapat menambah keamanan.
- Aplikasi perlu menangani kegagalan transaksi (error 1644 atau deadlock) dan mencobanya kembali.

8. Kesimpulan

1. Basis data praktikum_function_trigger dengan tiga tabel (produk, penjualan, log_stok), constraint, dan foreign key berhasil dibuat.
2. Function hitung_total, hitung_diskon, hitung_pajak, dan hitung_total_akhir menghasilkan nilai yang sesuai target pada semua pengujian.
3. Trigger validasi_penjualan berhasil menolak transaksi dengan jumlah nol atau melebihi stok serta mengisi total secara otomatis, sedangkan transaksi valid diterima.
4. Trigger update_stok_penjualan mengurangi stok dan mencatat log secara otomatis (Mouse 25 ke 23 ke 22, Keyboard 15 ke 12, Laptop 10 ke 8).
5. Trigger log_perubahan_stok dan validasi_harga_produk berhasil mencatat perubahan stok manual dan menolak harga nol atau negatif.
6. Gabungan function, trigger, dan constraint mengotomatisasi proses penjualan sekaligus menjaga integritas data. Perlu diperhatikan potensi pencatatan log ganda jika beberapa trigger mengubah tabel yang sama, serta penanganan konkurensi lewat transaksi dan penguncian baris.

Daftar Pustaka

Connolly, T., & Begg, C. (2015). Database Systems: A Practical Approach to Design, Implementation, and Management (6th ed.). Pearson.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.